

INDEX-CORRUPTION- RECONCILIATION

Index Corruption Reconciliation

Subject: LUMEN-INDEX-INTEGRITY.md (Report A) says the index is trustworthy. OLLAMA-NAN-WEDGE.md (Report B) infers it is contaminated. Which holds?

Database: ~/.local/share/lumen/21bf1507a8925bcf/index.db (118,620,160 bytes) **Project:** <ext-volume-host>/Projects/vasic
Investigated: 2026-08-27, 01:40–02:25 (+02:00) **Method:** read-only.
Every connection opened as `sqlite3.connect("file:<home>/~/.local/share/lumen/21bf1507a8925bcf/index.db?mode=ro", uri=True)` with `PRAGMA busy_timeout=300000`. No write was issued, no lumen subcommand was run, and **the ollama embedding API was never called** — a live lumen index (PID 2999211) was running throughout and a large probe could have wedged the backend it depends on.

VERDICT

(b) The index is CONTAMINATED.

758 of 35,717 stored vectors (2.12%) are byte-identical to one another while representing 695 distinct texts across 55 files. They are not NaN, not Inf, not all-zero, and every one carries an L2 norm of 1.000000 — so they pass **every** check Report A ran. They are nonetheless semantically meaningless: measured against known-good vectors for the same document sections in other languages, they score **0.25–0.47** where the good cross-language baseline is **0.967 median**.

The hypothesis under test is FALSIFIED. The corrupting condition did *not* require large chunks. Inside the corrupted range the **largest single chunk is 2,832 characters** — comfortably below the 4,000-char level at which Report B measured *zero* fence timeouts. Chunk size was never the protective factor it was assumed to be.

The operative quantity is the BATCH TOTAL, not the individual chunk. Lumen batches **32 chunks per embed request**. The 24 requests that produced the corruption carried **16,698–31,364 characters each** — 1.3× to 2.5× beyond the largest batch Report B ever validated as safe ($32 \times 400 = 12,800$ chars), and well past the 8,000-char single-request size at which Report B measured corruption on every attempt.

Question	Answer
Any NaN / Inf vectors?	0 — out of 35,717 decoded (100%, not a sample)
Any all-zero vectors?	0
Any wrong-dimension vectors?	0
Any norm outside 0.99–1.01?	0 (range 0.999999083 – 1.000000856)
Any degenerate vectors?	YES — 758 byte-identical clones of one vector
Chunk-length p99 / max (chars)	1,941 / 6,065
Chunks over 4,000 chars	4 (0.011%)
Chunks over 8,000 chars	0
Embed-request totals over 12,800 chars	244 of 1,107 (22.0%), max 52,877

Both prior reports are partly right, and both drew the wrong conclusion.

- **Report A’s measurements are correct and reproduce exactly.** Its *inference* — “no NaN therefore no corruption” — does not follow. Its four tests (NaN, Inf, all-zero, norm) are all blind to a stale-but-well-formed vector. Its one test that *would* have caught this, the distinctness check, was run on **blocks 1–4 only (4,096 vectors)**. The corruption lives in blocks 28–29.
- **Report B’s conclusion is right and its stated reason is wrong.** The index is contaminated, and the GPU path *did* return garbage at HTTP 200 — but not the all-zero signature Report B catalogued, and

not because chunks were large. Report B's own recommendation ("cap chunk size under ~4000 characters") would **not** have prevented this: every chunk involved was already under 2,832.

1. What size are the chunks Lumen sends to the embedder?

The raw text is not stored

Confirmed against the live schema (`SELECT type,name,sql FROM sqlite_master`):

```
CREATE TABLE chunks (  
  id          TEXT PRIMARY KEY,  
  file_path   TEXT NOT NULL REFERENCES files(path),  
  symbol      TEXT NOT NULL,  
  kind        TEXT NOT NULL,  
  start_line  INTEGER NOT NULL,  
  end_line    INTEGER NOT NULL  
);
```

There is no text, token-count, or byte-offset column. **The closest available proxy is reconstruction from disk by line span.**

The proxy, and its three limits

Convention verified empirically as **1-based inclusive**

(`lines[start_line-1:end_line]`): `.github/workflows/ci.yml` chunk name → line 51 → name: CI; chunk on → lines 53–57 → the on: block. Both match.

The embed envelope is taken to be `path\nsymbol\nbody`, which is Report A's finding, not mine — it validated it by re-embedding three chunks against the live model at cosine 0.98–0.99. I could not re-run that check (no API calls permitted this session), so the `path\n symbol\n` prefix (36–150 chars) is **inherited, not independently confirmed**. Both body-only and enveloped figures are given below so the distinction never matters to a conclusion.

Limit 1 — the proxy is invalid where the chunker slices below line granularity. Detected by finding (`file, start_line, end_line`) triples shared by more than one chunk: **2 such groups, 187 chunks**. The large one is decisive — `_analysis/github-vasic-digital.json` is an **88,206-byte minified single-line JSON array**. Lumen's StructuredChunker

walked it and produced exactly **185 chunks, one per array element** ([0]...[184]), all recorded as lines 1-1. The naive line proxy therefore reported 185 chunks of **88,114 characters each** — pure artifact. Parsing the file and measuring the true element spans gives **345–888 characters** (p50 468, max 888). All figures below use the corrected values.

Limit 2 — files must be unchanged since indexing. Checked: **0 of 944 indexed files are missing from disk**, and **0 chunks have an end_line beyond the current end of their file**. Working tree was clean at session start.

Limit 3 — characters are not tokens. Report B's thresholds are stated in characters but the underlying mechanism is token count against an 8192-token context. Character counts are a stable proxy only within one script; the corrupted region is Spanish and **Persian**, and Arabic-script text tokenizes far worse per character than Latin. See §6.

Distribution (n = 35,414 chunks, corrected)

Statistic	Body chars	Embed-text chars (path\nsymbol\nbody)
min	0	36
p25	29	81
median	43	99
mean	193	249
p75	73	141
p90	642	705
p95	1,111	1,177
p99	1,859	1,941
p99.9	2,179	2,273
max	5,594	6,065

The distribution is extremely bottom-heavy — a median chunk is a **single 43-character line** (YAML keys, JSON leaves, Markdown headings). The mass above 600 characters is prose Markdown sections and Python var aggregates.

2. How many chunks exceed Report B's thresholds?

Threshold	Body chars	Embed-text chars	Share
> 2,000	56	222	0.63%
> 4,000 (Report B: zero fence timeouts)	3	4	0.011%
> 8,000 (Report B: 6–7 fence timeouts, always corrupt)	0	0	0%
> 16,000	0	0	0%

The four chunks over 4,000 characters, in full:

Embed chars	Kind	Location
6,065	var	_tools/translate/cyrillize_sr.py:131-260
5,163	var	_tools/gen/translate_home.py:415-524
4,419	var	_tools/review_translation.py:372-442
4,083	var	_tests/tools/ glossary_protect_test.py:26-113

All four are Python var chunks whose symbol is a +-joined list of many identifiers — the signature of `chunker.deduplicateByExactRange` merging co-ranged declarations. **All four carry clean, unique, unit-norm vectors.**

So on the chunk axis the answer is: **the hypothesis' premise is true — nothing reaches 8,000 characters — and it is irrelevant, because that is not where the corruption came from.**

3. Does Lumen send a whole chunk as one request? No — it batches 32.

Established two ways, without calling the API.

From the binary (lumen-linux-amd64 v0.0.41, Go symbols intact in pcIntab):

```
github.com/ory/lumen/internal/embedder.(*Ollama).Embed
github.com/ory/lumen/internal/embedder.(*Ollama).embedBatch
github.com/ory/lumen/internal/config.
(*ConfigService).MaxChunkTokens
```

with the format string embedding batch starting at %d: %w, and the tunables LUMEN_MAX_CHUNK_TOKENS / max_chunk_tokens / LUMEN_EMBED_CTX.

None of these environment variables is set on the running indexer (read from /proc/2999211/environ), and there is no ~/.config/lumen/config.yaml, so stock defaults are in force.

From the log — every batch offset ever recorded in ~/.local/share/lumen/debug.log, across all projects:

```
0, 32, 64, 96, 128, 160, 192, 224, 256, 288, 352, 384, 416, 576,
992, 1024, 1440, 30752, 242624
GCD of the non-zero values = 32
```

Batch size is 32 chunks per /api/embed request, and the offset is a run-global chunk counter.

This changes which number matters

Reconstructing batches as consecutive runs of 32 in insertion order (vec_chunks_rowids.rowid, AUTOINCREMENT, so write order):

Statistic	Chars per embed request (32 chunks)
min	1,376
median	3,051
p75	3,501
p90	24,584
p95	28,890
p99	47,437
max	52,877

262 of 1,107 requests (23.7%) exceed 8,000 characters. 244 (22.0%) exceed 12,800 — the largest total Report B ever tested and cleared. The maximum is **4.1× that ceiling**.

The distribution is bimodal because the corpus is: batches of tiny YAML/JSON leaves land at ~3,000 chars; batches of prose Markdown sections land at 20,000–30,000.

Report B's condition (d) — `D1_gpu_batch32, 32 × 400 chars = 12,800 chars/request, 50 requests, zero fence timeouts` — is the only batch evidence in either report, and real indexing runs 1.3–4.1× above it.

4. Independent re-verification of Report A's core claim

Report A's documented `vec0` layout **still describes the file exactly**. Confirmed, not assumed:

Property	Expected	Measured (all 35 blocks)
<code>vec_chunks_chunks.size</code>	1024	1024 (all)
<code>length(Validity)</code>	<code>size/8 = 128</code>	128 (all)
<code>length(rowids)</code>	<code>size×8 = 8192</code>	8192 (all)
<code>length(vectors)</code>	<code>size×768×4 = 3,145,728</code>	3,145,728 (all)
Blocks violating the layout	0	0

Decode was written from scratch, not copied, and hardened with three checks Report A did not run:

1. Two independent read paths. (a)

`numpy.frombuffer(blob, '<f4').reshape(1024, 768)` in Python; (b) pure SQL `hex(substr(v.vectors, r.chunk_offset*3072+1, 3072))`. Both produce identical bytes for all 35,717 live slots.

2. Rowid cross-validation. The `chunk_id/chunk_offset` in

`vec_chunks_rowids` was checked against the `int64` sidecar inside `vec_chunks_chunks.rowids` at the same offset: **0 mismatches**.

Liveness per the validity bitmap was checked against presence in `vec_chunks_rowids`: **0 mismatches**.

3. **Layout falsification.** If the byte layout were being misread, unit norms would be meaningless. Reading block 1 three ways:

Interpretation	Fraction of first 64 rows with unit norm
float32 little-endian (documented)	1.000
float32 big-endian	0.000
float64 little-endian	0.000

Results — full decode, not a sample (n = 35,717, » the 300 requested)

Check	Result
NaN-containing vectors	0
Inf-containing vectors	0
All-zero vectors	0
Wrong dimensionality (≠768)	0
Norm outside 0.99–1.01	0
Validity-bitmap vs rowid-table mismatches	0
Sidecar-rowid mismatches	0

L2 norm: min 0.999999083 median 1.000000010 mean 1.000000006
p99.9 1.000000679 max 1.000000856 std 2.12e-07

Report A's stated numbers reproduce exactly, on a 19% larger index. On the four axes it tested, it was right and remains right.

The check Report A did not complete

Report A hashed **4,096 vectors from blocks 1–4** and found no duplicates. I hashed **all 35,717**:

distinct vector byte-patterns : 34,960 of 35,717
byte-identical groups : 1
largest group size : 758

A near-duplicate sweep (full $35,717 \times 35,717$ cosine, threshold ≥ 0.9999) returned **286,903 unordered pairs — exactly C(758,2)**. So the 758-clone group is the *only* duplication of any kind in the index; nothing else is even close.

5. The degenerate vectors, named

758 chunks share one identical 3,072-byte vector. They occupy a perfectly contiguous block of write order — `vec_chunks_rowids.rowid` **28205 through 28962, zero gaps, zero foreign rows inside the span.**

The vector itself looks entirely healthy: 768 distinct components, range -0.106 to $+0.107$, **L2 norm 1.000000083**. It is not a zero vector, not a NaN, not a constant fill. Nothing in Report A's test battery could have flagged it.

Proof that it is semantically wrong

(a) It is not a legitimate collision. The 758 chunks contain **695 distinct body texts** across **55 files** and **107 distinct symbols**, in two languages. A correct embedder cannot map 695 different documents to one vector.

(b) Cross-language control. The title chunk of `products/HelixQA.md` exists in seven language trees. Cosine between stored vectors:

	<code>_content</code>	<code>_ar</code>	<code>_de</code>	<code>_es</code>	<code>_fa</code>	<code>_fr</code>
<code>_content</code>	1.000	0.396	0.711	<i>0.249</i>	<i>0.249</i>	0.801
<code>_ar</code>	0.396	1.000	0.815	<i>0.471</i>	<i>0.471</i>	0.750
<code>_de</code>	0.711	0.815	1.000	<i>0.449</i>	<i>0.449</i>	0.932
<code>_es</code>	<i>0.249</i>	<i>0.471</i>	<i>0.449</i>	1.000	1.000	<i>0.384</i>
<code>_fa</code>	<i>0.249</i>	<i>0.471</i>	<i>0.449</i>	1.000	1.000	<i>0.384</i>
<code>_fr</code>	0.801	0.750	0.932	<i>0.384</i>	<i>0.384</i>	1.000

The healthy translations agree at 0.71–0.93. The Spanish and Persian entries agree with each other at **exactly 1.000** — because they are the same bytes — and with everything real at 0.25–0.47.

(c) Baseline. Across 53 randomly chosen cross-language pairs of the *same* section where both vectors are good: **median cosine 0.967** (p10 0.813, p90 0.993). The corrupt vectors sit far outside that distribution.

(d) It matches nothing in the corpus. Cosine of the bad vector against all 34,959 good vectors: **max 0.733, p99 0.647, median 0.427**. Its five nearest neighbours are `description_html` fields in `_analysis/gitlab-`

all.json — unrelated to any of its 695 supposed texts. It is also **not stale from a neighbouring request**: cosine against each of the 15 vectors written immediately before it ranges 0.239–0.423.

Affected files (55) — all 758 chunks

_content_fa/ is **100% corrupt** (508 of 508 indexed chunks).
_content_es/ is **54.6% corrupt** (250 of 458).

Chunks	File
4	_content_es/products/ HelixPlay.md (partial — 4 bad of 12; the other 8 are good)
18	_content_es/products/ HelixQA.md
12	_content_es/products/ HelixSkills.md
12	_content_es/products/ HelixSpecifier.md
13	_content_es/products/ HelixTerminator.md
14	_content_es/products/ HelixTrack.md
15	_content_es/products/ HelixTranslate.md
10	_content_es/products/ Herald.md
12	_content_es/products/ LLMOrchestrator.md
16	_content_es/products/ LLMProvider.md
15	_content_es/products/ LLMsVerifier.md
10	_content_es/products/Mail- Server-Factory.md
9	_content_es/products/ Parallels-Utils.md
10	_content_es/products/Qemu- Utils.md

Chunks	File
9	_content_es/products/ Server-Factory-Additional-Components.md
10	_content_es/products/ Server-Factory-Core-Framework.md
10	_content_es/products/ Vasic-Digital-Reusable-Module-Suite.md
10	_content_es/products/ VisionEngine.md
10	_content_es/products/ docs_chain.md
10	_content_es/products/ task_bridge.md
10	_content_es/sites/ milosvasic-ru.md
11	_content_es/sites/vasic-digital.md
4	_content_fa/docs/cover-letter.md
22	_content_fa/docs/cv.md
15	_content_fa/docs/ portfolio.md
13	_content_fa/products/ Catalogizer.md
10	_content_fa/products/ Courses-Creator.md
10	_content_fa/products/ DocProcessor.md
12	_content_fa/products/ Helix-Flow.md
17	_content_fa/products/ HelixAgent.md
12	_content_fa/products/ HelixBuilder.md
22	_content_fa/products/ HelixCluster.md

Chunks	File
19	_content_fa/products/ HelixCode.md
21	_content_fa/products/ HelixConstitution.md
13	_content_fa/products/ HelixGitpx.md
19	_content_fa/products/ HelixLLM.md
19	_content_fa/products/ HelixMemory.md
19	_content_fa/products/ HelixOTA.md
16	_content_fa/products/ HelixPlay.md
24	_content_fa/products/ HelixQA.md
16	_content_fa/products/ HelixSkills.md
13	_content_fa/products/ HelixSpecifier.md
19	_content_fa/products/ HelixTerminator.md
22	_content_fa/products/ HelixTrack.md
20	_content_fa/products/ HelixTranslate.md
11	_content_fa/products/ Herald.md
21	_content_fa/products/ LLMOrchestrator.md
21	_content_fa/products/ LLMProvider.md
21	_content_fa/products/ LLMsVerifier.md
11	_content_fa/products/Mail- Server-Factory.md
10	

	Chunks	File
		_content_fa/products/ Parallels-Utills.md
	10	_content_fa/products/Qemu- Utills.md
	10	_content_fa/products/ Server-Factory-Additional- Components.md
	6	_content_fa/products/ Server-Factory-Core- Framework.md
	10	_content_fa/products/ Vasic-Digital-Reusable- Module-Suite.md

Boundaries: first corrupt chunk 3d53640439a9dda2 = _content_es/products/HelixPlay.md:59-65 (Qué lo hace innovador); last = 61d4b33089a1f563 = _content_fa/products/Vasic-Digital-Reusable-Module-Suite.md:65-77. The chunk written immediately before, rowid 28204 = _content_es/products/HelixPlay.md:55-58, is **clean and unique** — the corruption starts *mid-file*, which is a batch boundary, not a file boundary.

These will NOT self-heal

Every one of the 55 files carries a **non-empty hash in the files table**. Unlike the 175 empty-hash placeholder rows Report A analysed — which are genuinely self-healing — these files are recorded as **successfully indexed**. Lumen will skip them on every subsequent run unless their content changes on disk. **Resuming the index does not fix this. Only a rebuild, or touching the 55 files, will.**

6. What actually happened

The corruption correlates tightly with a kernel GPU fault, and the timing matches Report B's mechanism precisely.

```
22:25:34 kernel: Fence expiration time out
i915-0000:00:02.0:ollama[4101609]:2388c!    (x3)
...      ~8m40s of HTTP 200 responses carrying one repeated
vector
22:34:14 kernel: i915 [drm] Resetting rcs0 for preemption time
```

```
out
22:34:14 kernel: i915 [drm] ollama[4101609] context reset due to
GPU hang
22:34:15 kernel: i915 [drm] GPU HANG: ecode 12:1:859ffffb, in
ollama [4101609]
22:34:17 lumen: indexing failed (embed error HTTP 400: ... Post
.../embedding: EOF)
```

Runner PID 4101609 was the GPU-offloaded runner serving the 20:38:45 indexing run. It took three fence timeouts at 22:25:34, then served **~24 further batches at HTTP 200** with a degraded Vulkan context returning the same result buffer every time, and was finally reset out from under ollama by the kernel at 22:34:14 — which killed the run three seconds later. Everything committed in that window is the 758-chunk block.

This is **Report B’s mechanism exactly** (§4.1: “the dispatch is abandoned... ollama reads that buffer anyway”), with a **fourth output mode Report B did not catalogue**: not zeros, not NaN, not a slightly-wrong vector, but *the same stale valid-looking vector repeated indefinitely*. Of Report B’s three documented modes, only zerovec and NaN are detectable by a norm check; this one is not detectable by any per-vector test at all — it is only visible by comparing vectors to each other.

Why these batches and not others

The corrupted requests carried **16,698–31,364 characters each** (median 20,265), against a whole-index median of 3,051. Two compounding reasons:

1. **_content_es/ and _content_fa/ are long prose Markdown**, so 32 chunks per request is 20–30 kB rather than 3 kB.
2. **_content_fa is Persian (Arabic script)**, which tokenizes far worse per character than Latin text in a BPE tokenizer trained mainly on code and English. A 20 kB Persian batch plausibly exceeds the 8192-token batch the runner is loaded with (BatchSize:8192 KvSize:8192) by a wide margin. *This is a hypothesis consistent with the data, not a measurement — see §8.*

Note that **_content_ar/** (also Arabic script) came through clean, so script alone is not sufficient. The full trigger is presumably total tokens per dispatch, which is `batch_size × mean_tokens_per_chunk` — and Lumen fixes the first factor at 32 regardless of the second.

7. Measured headroom against Report B's thresholds

Axis	Measured	Report B's clean level	Report B's corrupting level	Headroom
Single chunk, p99	1,941 ch	4,000	8,000	2.1× under clean — safe
Single chunk, max	6,065 ch	4,000	8,000	1.5× OVER clean , 1.3× under corrupting — inside Report B's un-bisected bracket
Chunks > 8,000	0	—	—	no exposure on this axis
Embed request, median	3,051 ch	12,800	(untested)	4.2× under — safe
Embed request, p90	24,584 ch	12,800	(untested)	1.9× OVER
Embed request, max	52,877 ch	12,800	(untested)	4.1× OVER
Requests over 12,800 ch	244 of 1,107 (22.0%)	—	—	one fifth of all traffic is beyond validated-safe
Observed corrupting batch	16,698–31,364 ch	—	—	corruption occurred here, with max

Axis	Measured	Report B's clean level	Report B's corrupting level	Headroom
				chunk 2,832

On the chunk axis the index has real headroom and is not the problem. On the batch axis it is routinely and substantially outside anything that has been shown to be safe, and that is where it broke.

8. What remains unproven

1. **The path\symbol\nbody embed envelope is inherited from Report A, not re-verified.** Confirming it requires re-embedding stored chunks against the live model, which was out of scope (no API calls). It shifts every chunk figure by a constant 36–150 characters and changes no conclusion.
2. **Batch composition is reconstructed, not observed.** I modelled requests as consecutive runs of 32 in `vec_chunks_rowids` insertion order. Batch size 32 is solid (binary symbols
 - GCD of every logged offset), but whether batches are packed strictly in that order, and whether they reset at file boundaries, is inferred. The corrupted range is an empirical anchor that does not depend on the model: 758 chunks totalling 515,169 characters were corrupted together with no chunk exceeding 2,832 characters.
3. **The Persian-tokenization explanation is a hypothesis.** I did not tokenize anything — no tokenizer for `jina-embeddings-v2-base-code` was invoked. `_content_ar/` indexed cleanly, which the hypothesis does not by itself explain.
4. **The causal chain “GPU fence timeout → stale buffer → repeated vector” is inference from correlation.** The timing is tight (fault at 22:25:34, runner destroyed 22:34:14, run died 22:34:17, corrupt block contiguous and bounded by exactly that run) and it matches Report B’s independently reproduced mechanism. But I did not reproduce it — deliberately, since probing would have wedged the live job. **An alternative explanation I cannot exclude: a buffer-reuse bug in Lumen’s own `embedBatch` response handling.** The observable effect is identical, and the remediation for the index is identical.

5. **Whether the 4 chunks over 4,000 characters are themselves safe is not established** — they happen to carry clean vectors today, but 6,065 sits inside the 4,000–8,000 bracket that Report B explicitly declined to bisect (its §9.3).
 6. **The index is still being written, and everything above is a snapshot.** The live lumen_index (PID 2999211) has been running since 01:36:31. vec_chunks_rowids held 35,717 rows at 02:05 and again at 02:22 (a ~17-minute pause), then 35,976 by 02:29 as it resumed — currently working through ai_interviewing/. The §1–§7 figures are the 02:22 snapshot. A spot check of the **400 newest vectors at 02:29 found 0 duplicates, 0 NaN, 0 all-zero, norms 0.9999993–1.0000007** — clean so far, but the backend is still on the faulty GPU path (§9), so **corruption arising after 02:29 is not covered by this report.**
 7. **No claim is made about the 8 uncorrupted chunks of _content_es/products/HelixPlay.md beyond the tests run** — they are unique and unit-norm, which is necessary but, as this whole report shows, not sufficient.
-

9. Consequences

The corruption mechanism is live right now. As of 02:22 today:

- /etc/sysconfig/ollama **does not exist** — Report B's Tier 3 fix is not applied.
- ollama ps reports ordis/jina-embeddings-v2-base-code ... 100% GPU, and the runner loaded at 01:36:31 logged load_tensors: offloaded 13/13 layers to GPU, Vulkan0 model buffer size = 216.53 MiB.
- project_meta.embedding_model is still ordis/jina-embeddings-v2-base-code — the CPU-pinned tag jina-embeddings-code-cpu exists but **Lumen is not using it.**
- **236 i915 fence timeouts have been logged since midnight**, including 7 at 01:35 and a GPU-hang/context-reset sequence at 00:41.

Two indexing runs failed today (00:46:55, 01:35:33) and the currently running one is on the same faulty path that produced the 758 bad vectors.

Recommended, in order:

1. **Apply Report B's fix before any further indexing.** Point Lumen at the CPU-pinned tag jina-embeddings-code-cpu (Tier 2, no restart,

already validated in Report B §8), or create `/etc/sysconfig/ollama` with `GGML_VK_VISIBLE_DEVICES=-1` and restart the service while no job is running.

2. **Then rebuild clean — Report B was right on this point, Report A wrong.** Resuming cannot repair the 758 vectors: their files are recorded as successfully indexed. A narrower alternative is to touch the 55 files listed in §5 so their hashes change and Lumen re-processes them; that recovers 2.12% of the index without discarding the other 97.88%.
 3. **Add a duplicate-vector check to whatever verifies the index.** Norm, NaN and zero checks all pass on this corruption. Byte-hashing every vector and asserting near-total distinctness is cheap (~4 s for 35,717 vectors) and is the only test here that caught it.
 4. **Reconsider the batch size.** `LUMEN_MAX_CHUNK_TOKENS` caps individual chunks, but the dispatch is 32 chunks wide and nothing caps the total. Report B's advice to cap *chunk* size would not have prevented this incident.
-

Appendix: reproducing this report

All read-only. Chunk-size reconstruction:

```
import sqlite3, os
con = sqlite3.connect("file:<home>/local/share/lumen/"
                      "21bf1507a8925bcf/index.db?mode=ro",
                      uri=True, timeout=300)
con.execute("PRAGMA busy_timeout=300000")
rows = con.execute("""SELECT r.rowid, c.file_path, c.symbol,
                          c.start_line, c.end_line
                      FROM vec_chunks_rowids r JOIN chunks c ON
                          c.id = r.id
                      ORDER BY r.rowid""").fetchall()
# body = "\n".join(open(path).read().split("\n")
#                  [start_line-1:end_line]) # 1-based inclusive
# NOTE: invalid where several chunks share one (file,start,end) -
#       see §1, Limit 1
```

Duplicate detection — the check that settles it, via a pure-SQL read path:

```
SELECT hex(substr(v.vectors, r.chunk_offset*3072 + 1, 3072)) AS
       h, COUNT(*) AS n
FROM vec_chunks_rowids r
JOIN vec_chunks_vector_chunks00 v ON v.rowid = r.chunk_id
GROUP BY h HAVING n > 1 ORDER BY n DESC;
-- returns exactly one row: n = 758
```

Naming the affected chunks:

```
SELECT r.rowid, c.file_path, c.symbol, c.start_line, c.end_line
FROM vec_chunks_rowids r JOIN chunks c ON c.id = r.id
WHERE r.rowid BETWEEN 28205 AND 28962
ORDER BY r.rowid;
```

Confirming they will not be re-indexed:

```
SELECT COUNT(*) FROM files
WHERE hash <> '' AND path IN (SELECT DISTINCT c.file_path FROM
    chunks c
    JOIN vec_chunks_rowids r ON r.id = c.id WHERE r.rowid BETWEEN
        28205 AND 28962);
-- 55 (i.e. all of them)
```

Batch size, from the log:

```
grep -aoE 'batch starting at [0-9]+' ~/.local/share/lumen/
    debug.log \
    | grep -oE '[0-9]+' | sort -nu          # GCD of non-zero values =
    32
```

GPU fault correlation:

```
journalctl -k --since "2026-08-26 22:20" --until "2026-08-26
    22:40" -o short-iso \
    | grep -i 'fence expiration\|GPU HANG\|context reset'
```